Danny's Diner — SQL Case Study #1

8 Week SQL Challenge by Danny Ma

Q1. What is the total amount each customer spent at the restaurant?

SQL Query:

```
SELECT customer_id, SUM(price) AS total_amount
FROM sales
INNER JOIN menu USING (product_id)
GROUP BY customer_id;
```

Output:

	customer_id character varying (1)	total_amount bigint
1	B	74
2	C	36
3	A	76

Q2. How many days has each customer visited the restaurant?

SQL Query:

```
SELECT customer_id, COUNT(DISTINCT(order_date)) AS time_visit
FROM sales
INNER JOIN menu USING (product_id)
GROUP BY customer_id;
```

Output:

	customer_id character varying (1)	time_visit bigint
1	A	4
2	B	6
3	C	2

Q3. What was the first item from the menu purchased by each customer?

SQL Query:

```
SELECT DISTINCT ON (customer_id) customer_id, order_date, product_name AS first_dish
FROM sales JOIN menu USING (product_id)
ORDER BY customer_id, order_date;
```

Output:

	customer_id character varying (1)	order_date date	first_dish character varying (5)
1	A	2021-01-01	curry
2	B	2021-01-01	curry
3	C	2021-01-01	ramen

Q4. What is the most purchased item on the menu and how many times was it purchased by all customers?

SQL Query:

```
SELECT product_name AS dish, COUNT(product_id) AS times_eaten
FROM sales INNER JOIN menu USING(product_id)
GROUP BY (product_id, product_name)
ORDER BY times_eaten DESC
LIMIT 1;
```

Output:

	dish character varying (5)	times_eaten bigint
1	ramen	8

Q5. Which item was the most popular for each customer?

SQL Query:

```
WITH ranked_orders AS (
    SELECT
        customer_id,
        product_name,
        COUNT(*) AS time_ordered,
        RANK() OVER (PARTITION BY customer_id ORDER BY COUNT(*) DESC) AS rk
    FROM sales
    JOIN menu USING(product_id)
    GROUP BY customer_id, product_name
)
SELECT customer_id, product_name, time_ordered
FROM ranked_orders
WHERE rk = 1
ORDER BY customer_id;
```

Output:

	customer_id character varying (1)	product_name character varying (5)	time_ordered bigint
1	A	ramen	3
2	B	sushi	2
3	B	curry	2
4	B	ramen	2
5	C	ramen	3

Q6. Which item was purchased first by the customer after they became a member?

SQL Query:

```
SELECT
DISTINCT ON (customer_id) customer_id,
order_date,
product_name
FROM sales
JOIN members USING (customer_id)
JOIN menu USING (product_id)
WHERE order_date >= join_date
ORDER BY customer_id, order_date;
```

Output:

	customer_id character varying (1)	order_date date	product_name character varying (5)
1	A	2021-01-07	curry
2	B	2021-01-11	sushi

Q7. Which item was purchased just before the customer became a member?

SQL Query:

```
SELECT
DISTINCT ON (customer_id) customer_id,
order_date,
product_name
FROM sales
JOIN members USING (customer_id)
JOIN menu USING(product_id)
WHERE order_date <= join_date
ORDER BY customer_id, order_date;
```

Output:

	customer_id character varying (1)	order_date date	product_name character varying (5)
1	A	2021-01-01	sushi
2	B	2021-01-01	curry

Q8. What is the total items and amount spent for each member before they became a member?

SQL Query:

```
SELECT
customer_id,
COUNT(product_id) AS total_items,
SUM(price) AS total_amount_spent
FROM members
JOIN sales USING (customer_id)
JOIN menu USING(product_id)
WHERE order_date >= join_date
GROUP BY customer_id;
```

Output:

	customer_id character varying (1)	total_items bigint	total_amount_spent bigint
1	B	3	34
2	A	4	51

Q9. If each $1 spent equates to 10 points and sushi has a 2x points multiplier — how many points would each customer have?

SQL Query:

```
SELECT
    s.customer_id,
    SUM(
        CASE
            WHEN m.product_name = 'sushi' THEN price * 10 * 2
            ELSE price * 10
        END
    ) AS points
FROM sales s
JOIN menu m USING(product_id)
GROUP BY s.customer_id
ORDER BY s.customer_id;
```

Output:

	customer_id character varying (1)	points bigint
1	A	860
2	B	940
3	C	360

Q10. In the first week after a customer joins the program (including their join date) the entire menu earns 2x points on all items — how many points do customer A and B have at the end of January 2021?

SQL Query:

```
SELECT
customer_id,
SUM(CASE
        WHEN s.order_date BETWEEN m.join_date
             AND m.join_date + INTERVAL '6 days'
            THEN price * 20
        WHEN s.order_date < m.join_date
             AND mn.product_name = 'sushi'
            THEN price * 20
        WHEN s.order_date > m.join_date + INTERVAL '6 days'
             AND mn.product_name = 'sushi'
            THEN price * 20
        ELSE price * 10
    END) AS total_points
FROM sales s
JOIN menu mn USING (product_id)
JOIN members m USING (customer_id)
WHERE s.order_date >= DATE '2021-01-01'
  AND s.order_date <  DATE '2021-02-01'
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

	customer_id character varying (1)	total_points bigint
1	A	1370
2	B	820